

Path Planning Optimization with A* Search Algorithm

Evan Litzer

Abstract—This report entails and explains how the A* algorithm fulfills the assignment's requirements by searching for and discovering the shortest path for each of the attached grid files. Through testing and analysis, this research demonstrates that the A* algorithm is completely effective in generating optimal paths while maintaining reasonable execution time and memory-use.

I. INTRODUCTION

Path-planning in this assignment describes an algorithm that begins at a starting point and reaches an ending point by avoiding obstacles in the grid. The A* algorithm is one of the most efficient path-finding algorithms, as it combines both the Dijkstra's and Greedy Best Search through utilization of a heuristic function. This function prioritizes paths with lower costs to the end while ensuring that it maintains optimality.

This report discovers how the A* algorithm searches and finds the shortest path with any provided grid file. The report is structured as follows: Section II discusses the search algorithm methodology,--appointing the implementation of the A* searching algorithm. Section III presents the results and analysis of the function, evaluating its efficient and optimal performance. Section IV concludes the findings of the project and explores other improvements or alternative approaches that can alter the searching algorithm.

II. METHODOLOGY & IMPLEMENTATION OF A*

A. Overview of the A* Algorithm

The A* algorithm was advised in class as the best algorithm for path-planning, and it proves its effectiveness in this assignment. The algorithm finds the shortest path from the grid file's starting and ending point whilst avoiding any obstacle points in the grid. Using its cost function, it determines whether a node's current and future expense belongs in the grid's shortest path from start to goal position.

B. Implementation of A* Algorithm

In this assignment, the A* algorithm is found within the planner.py file represented by its own function. The function passes in the grid as a list along with the integer start and end representing the grid's start and end positions. In the grid-file the algorithm inspects and builds upon, 0 represents walkable space while 1 represents an obstacle. The grids are passed into the searching function through main.py, which reads in the grid files and calls the planner.py's plan_path function, which calls the searching algorithm with the grid and its information.

The algorithm is executed by first initializing the open set by adding the grid's starting position to a priority queue min-heap, which is sorted based on the total cost function $f(n)$ of the node. The algorithm then selects the node with the lowest cost and explores every neighboring node accessible from the directional array, which includes up, down, left, right, and diagonal directives. The cost to reach each neighbor (along with its cost to the end point) is computed through $f(n)$, and the node is added to the open set if it envisions a better path than previously set. Once the endpoint of the grid is reached, the algorithm backtracks to its starting point to then reconstruct the optimal path and its associated cost.

III. A* ALGORITHM MATH

A. Graphs & Pathing

The 2D grid is developed through the graph system with the equation $G(V, E)$, where V is the set of possible grid cells (formerly known as nodes) while E is the set of edges $\subseteq V \times V$ representing movements between nodes. All of the nodes $v \in V$ are either walkable (0) or an obstacle (1). The algorithm attempts to find an optimal path P from start node s to the end node g . The path P is defined as $P = \{v_0, v_1, \dots, v_n\}$ where $v_0 = s$ and $v_n = g$. The total cost $f(v)$ of the path is minimized through the A* algorithm.

B. Cost Function & Heuristics

A* evaluates every neighboring node n using the cost function: $f(n) = g(n) + h(n)$. In this function, $g(n)$ is the actual cost from starting position to n . This is calculated through $g(n) = g(p) + d(p, n)$, where p is the parent node of n and $d(p, n)$ is the movement cost from parent node to n . Note that all movement costs to neighbors are equal, both diagonal and horizontal/vertical equal one. $H(n)$ is the heuristic estimate of the remaining cost to reach the goal position, which is calculated through $h(n) = \max(|x_n - x_g|, |y_n - y_g|)$. This finds the absolute difference from the node to the end point, which the maximum value indicates how many steps (at most) are required to reach the goal.

The neighboring node with the smallest cost $f(n)$ through $n^* = \operatorname{argmin}(n \in O) f(n)$ where O is the set of open nodes. For all of the neighbors n' of n , if $g(n') > g(n) + d(n, n')$, then $g(n') = g(n) + d(n, n')$ and $f(n') = g(n') + h(n')$ are updated and n' is pushed into the priority queue.

C. Path Validity Checks

The path P is valid if it has complete connectivity, obstacle escapism, and computing grid bounds. For connectivity, $\forall i d(n_i, n_{i+1}) \leq 1$. This ensures each step that is made is to a neighboring node. For obstacle avoidance, $\forall n \in P, n \notin Ob$.

Where Ob is the set of obstacle positions, this ensures no obstacle is included in the path. Finally regarding grid bounds, $\forall n = (x, y) \in P$, both conditions $0 \leq x < M$ and $0 \leq y < N$ assures the path remains within the grid boundaries. The algorithm ends its search when $n^* = g$, which tells the algorithm that the goal has been reached. It also ends when no valid paths remain in the open set.

IV. PROOF OF CONSISTENCY & ADMISSABILITY

A. Admissibility of A^*

Admissibility with heuristic function $h(n)$ is applicable if the function never overestimates the true cost $h^*(n)$, which translates to the function calculation the shortest distance from the current node n to the goal node g . This is represented by the statement: $\forall n \in V, h(n) \leq h^*(n)$.

The A^* algorithm warrants the optimal solution if the heuristic function is admissible. My algorithm's heuristic function achieves this, as its defined as $h(n) = \max(|x_n - x_g|, |y_n - y_g|)$. This formula provides the shortest path for each of the grid files because of its constant urge to utilize diagonal movement. This diagonal movement allows the algorithm to reduce both the (x, y) coordinates with equivalent cost to a vertical or horizontal move. The minimum number of steps indicated by the biggest difference in either x or y values between start and goal nodes is correctly estimated by this heuristic function.

After the heuristic calculation, A^* only chooses to expand on the node with lowest cost in $f(n) = g(n) + h(n)$. Since $h(n)$ never overestimates the minimum number of steps required to reach the goal from any given node, the algorithm will never exclude the shortest path from its considerations. A^* prioritizes and chooses the path with the lowest cost, therefore it will always reach the most optimal and lowest-cost path.

Therefore the condition $\forall n \in V, h(n) \leq h^*(n)$ is fulfilled, and A^* is proven to be admissible.

B. Consistency of A^*

The heuristic function of an algorithm is proven to be consistent if it satiates this condition. For each node n and its neighbor n' , estimated cost is equivalent to $h(n) \leq d(n, n') + h(n')$. In this equation, $d(n, n')$ is the cost of movement from n to its neighboring node n' , while $h(n')$ is the heuristic cost of the neighboring node.

$H(n) = \max(|x_n - x_g|, |y_n - y_g|)$ and $h(n') = \max(|x_{n'} - x_g|, |y_{n'} - y_g|)$ for the current and neighboring nodes respectively. All movement costs to neighboring nodes are equal to one, so movement from n to n' will decrease the absolute values of the x or y differences by one. Therefore, $h(n) - h(n') \leq 1$. Thus the consistency condition $h(n) \leq h(n') + d(n, n')$ is satisfied, as the cost from n is never greater than the cost from n' added with the cost to get from n to n' .

Therefore the heuristic function is consistent, as A^* remains efficient and is ensured to find the optimal path. This also means that A^* never repeats checking a node and its path. By definition, this also reciprocates that the heuristic function is also admissible, but it has already been proven.

V. CONCLUSIONS

This report completely examined my A^* algorithm and its heuristic function as an optimal and admissible solution to the project's path-planning assignment. It has demonstrated through adamant testing, along with logical proofing, that it is able to find the shortest path in the provided grid files while avoiding obstacles and higher costs.

My implementation further proves theorem three that was discussed in class, simply stating that A^* is the best algorithm. Therefore, the optimal algorithm is A^* and it fulfills all required conditions in discovering the shortest path of all one-hundred grids.